ДЕПАРТАМЕНТ ОБРАЗОВАНИЯ И НАУКИ ГОРОДА МОСКВЫ
ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ГОРОДА МОСКВЫ
«КОЛЛЕДЖ СВЯЗИ №54»
ИМЕНИ П.М. ВОСТРУХИНА

ПРАКТИЧЕСКАЯ РАБОТА № 2

МДК.09.01

Проектирование и разработка веб-приложений

Тема: «Базовые CRUD-операции через консоль»

Выполнила

ст. Гр. 4-ИСП9-46

Миронова Анна Андреевна

Москва, 2026

СОДЕРЖАНИЕ

Цель работы 3

Реализованный функционал 3

Запуск 3

Средства разработки 3

Анализ сложности 3

Результаты выполнения 4

Ответы на контрольные вопросы 4

Проверка результатов 6

Заключение 7

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ И ИНТЕРНЕТ-ИСТОЧНИКОВ 8

Листинги программ 9

Цель работы

Освоить: базовые crud-операции через консоль.

Условие: практическая работа №2.

Реализованный функционал

Реализовано меню: список, добавление, завершение задачи, удаление и выход. Все пользовательские значения передаются через placeholders ?. Ввод ID проверяется до преобразования; отсутствующий ID возвращает 0 изменённых строк. Соединения закрываются в finally, ошибки транзакций вызывают roll-back.

Запуск

```
python web/practice_02/crud_console.py
```

Средства разработки

Python 3.14.6, Flask 3.1.2, SQLite 3.53.3. Проверки выполнены в Arch Linux/WSL. Запуск в POCA и Python 3.8 отдельно не проверялся.

Порядок запуска: инструкция.

Анализ сложности

Пусть n — число строк, L — длина названия. Список ORDER BY id: O(n) времени и O(n) памяти результата. INSERT, UPDATE и DELETE по первичному ключу: O(log n) для поиска/изменения B-дерева; обработка названия добавляет O(L). Дополнительная память одиночной операции O(L).

Результаты выполнения

Результат проверки показан на рисунке 1.

```
$ printf $'2\\n\320\237\320\276\320\264\320\263\320\276\321\202\320\276\320\262\320\270\321\202\321\214 \320\276\321\202\321
\207\321\221\321\202\\n1\\n3\\n1\\n1\\n4\\n1\\n1\\n0\\n'
$ python web/practice_02/crud_console.py

1 — Список; 2 — Добавить; 3 — Выполнить; 4 — Удалить; 0 — Выход
Команда: Название: Создана задача # 1

1 — Список; 2 — Добавить; 3 — Выполнить; 4 — Удалить; 0 — Выход
Команда: #1 | Подготовить отчёт | new

1 — Список; 2 — Добавить; 3 — Выполнить; 4 — Удалить; 0 — Выход
Команда: ID: Изменено строк: 1

1 — Список; 2 — Добавить; 3 — Выполнить; 4 — Удалить; 0 — Выход
Команда: #1 | Подготовить отчёт | done

1 — Список; 2 — Добавить; 3 — Выполнить; 4 — Удалить; 0 — Выход
Команда: ID: Изменено строк: 1

1 — Список; 2 — Добавить; 3 — Выполнить; 4 — Удалить; 0 — Выход
Команда: Список пуст

1 — Список; 2 — Добавить; 3 — Выполнить; 4 — Удалить; 0 — Выход
Команда:
Код завершения: 0
```

Рисунок 1 – Добавление, чтение, завершение и удаление задачи в консоли

Ответы на контрольные вопросы

1. Почему SELECT не требует commit, а INSERT требует?

SELECT не меняет данные. INSERT в используемом режиме открывает транзакцию; commit фиксирует изменения. Незавершённая транзакция при закрытии откатывается.

2. Что вернёт fetchone без строк?

None.

3. fetchone и fetchall: когда использовать?

fetchone возвращает одну следующую строку или None; удобен при поиске по уникальному ID. fetchall возвращает список оставшихся строк; удобен для небольшого полного списка.

4. Почему (title,) — кортеж?

Кортеж из одного элемента создаётся запятой. Скобки без запятой лишь группируют выражение.

5. Что вернёт rowcount при отсутствующем ID?

0: ни одна строка не подошла под WHERE.

6. Зачем проверять isdigit перед int?

Чтобы заранее отклонить недопустимый ввод и не получить ValueError. В решении также проверяются ASCII-цифры и положительность: некоторые Unicode-символы проходят isdigit, но не преобразуются int.

7. Как placeholders защищают от SQL-инъекции?

SQL-текст и значения передаются отдельно. Вредоносная строка становится содержимым title и не интерпретируется как команда DROP TABLE.

Проверка результатов

Результат проверки показан на рисунке 2.

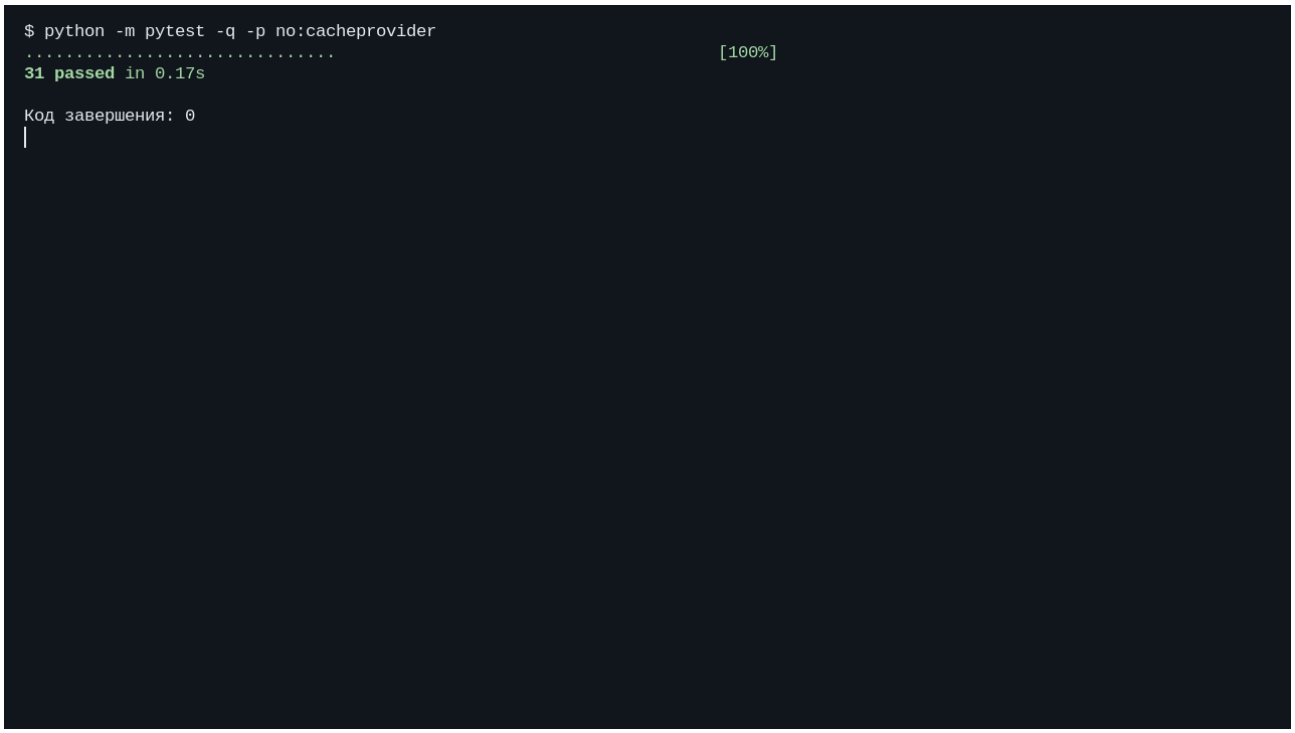


Рисунок 2 – Результат запуска автоматических Python-тестов

Заключение

Разработана консольная программа для добавления, просмотра, завершения и удаления задач. Пользователь выбирает действие в меню, а запись для изменения определяется по идентификатору. После добавления и изменения данные фиксируются в SQLite; повторное чтение показывает сохранённое состояние. Обращение к отсутствующему идентификатору обрабатывается без аварийного завершения.

Проверены неверный ввод идентификатора, пустое название и передача строки с SQL-командой. Параметризованные запросы сохраняют такую строку как значение поля, сохраняя таблицу tasks. При работе с транзакциями INSERT, UPDATE и DELETE требуют фиксации изменений. SELECT используется для чтения и не меняет содержимое базы. Консольная версия даёт основу для переноса этих операций в HTTP-обработчики.

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ И ИНТЕРНЕТ-ИСТОЧНИКОВ

1. Гниденко, И.Г. Технологии и методы программирования [Текст]: учебное пособие для вузов / И.Г. Гниденко, Ф.Ф. Павлов, Д.Ю. Федоров. – 2-е изд., перераб. и доп. – Москва: Юрайт, 2024. – 248 с.
2. Жуков, Р.А. Язык программирования Python: практикум [Текст]: учебное пособие / Р.А. Жуков. – Москва: ИНФРА-М, 2026. – 216 с.
3. Федоров, Д.Ю. Программирование на Python [Текст]: учебное пособие для вузов / Д.Ю. Федоров. – 6-е изд., перераб. и доп. – Москва: Юрайт, 2025. – 187 с.
4. Фёдорова, Г.Н. Разработка, внедрение и адаптация программного обеспечения отраслевой направленности [Текст]: учебное пособие / Г.Н. Фёдорова. – Москва: КУРС: ИНФРА-М, 2024. – 336 с.
5. Чернышев, С.А. Основы программирования на Python [Текст]: учебное пособие для вузов / С.А. Чернышев. – 2-е изд., перераб. и доп. – Москва: Юрайт, 2024. – 349 с.

Электронные ресурсы

6. Документация веб-фреймворка Flask [Электронный ресурс]: https://flask.palletsprojects.com/en/stable/ (дата обращения 10.11.2026).
7. Официальное руководство по языку Python и модулю sqlite3 [Электронный ресурс]: https://docs.python.org/3/library/sqlite3.html (дата обращения 10.11.2026).
8. Справочник по реляционной СУБД SQLite [Электронный ресурс]: https://www.sqlite.org/docs.html (дата обращения 10.11.2026).

Листинги программ

Web/practice_02/crud_console.py

```
import os
import sqlite3
from pathlib import Path

DB_PATH = Path(os.environ.get("TASK_DB", Path(__file__).with_name("app.db")))


def init_db(db_path=DB_PATH):
    conn = sqlite3.connect(db_path)
    try:
        conn.execute(
            "CREATE TABLE IF NOT EXISTS tasks (id INTEGER PRIMARY KEY, title TEXT NOT
NULL, status TEXT DEFAULT 'new')"
        )
        conn.commit()
    finally:
        conn.close()


def query(sql, params=(), db_path=DB_PATH, read=False):
    conn = sqlite3.connect(db_path)
    try:
        cursor = conn.execute(sql, params)
        if read:
            return cursor.fetchall()
        conn.commit()
        return cursor.lastrowid if sql.startswith("INSERT") else cursor.rowcount
    except sqlite3.Error:
        conn.rollback()
        raise
    finally:
        conn.close()


def list_tasks(db_path=DB_PATH):
    return query(
        "SELECT id,title,status FROM tasks ORDER BY id", db_path=db_path, read=True
    )


def add_task(title, db_path=DB_PATH):
    title = title.strip()
    if not title:
        raise ValueError("Название не может быть пустым")
    return query("INSERT INTO tasks(title) VALUES (?)", (title,), db_path)


def complete_task(task_id, db_path=DB_PATH):
    return query("UPDATE tasks SET status='done' WHERE id=?", (task_id,), db_path)


def delete_task(task_id, db_path=DB_PATH):
    return query("DELETE FROM tasks WHERE id=?", (task_id,), db_path)
```

```
def main():
    init_db()
    while True:
        print("\n1 — Список; 2 — Добавить; 3 — Выполнить; 4 — Удалить; 0 — Выход")
        try:
            choice = input("Команда: ").strip()
            if choice == "0":
                return
            if choice == "1":
                rows = list_tasks()
                for row in rows:
                    print(f"#{row[0]} | {row[1]} | {row[2]}")
                if not rows:
                    print("Список пуст")
            elif choice == "2":
                print("Создана задача #", add_task(input("Название: ")))
            elif choice in ("3", "4"):
                value = input("ID: ").strip()
                if not value.isascii() or not value.isdigit() or int(value) < 1:
                    print("Введите положительный целый ID")
                    continue
                result = (complete_task if choice == "3" else delete_task)(int(value))
                print("Изменено строк:", result)
            else:
                print("Неизвестная команда")
        except (sqlite3.Error, ValueError) as exc:
            print("Ошибка:", exc)
        except (EOFError, KeyboardInterrupt):
            return


if __name__ == "__main__":
    main()
```

Web/practice_02/init_db.py

```
import os
import sqlite3
from pathlib import Path

DB_PATH = Path(os.environ.get("TASK_DB", Path(__file__).with_name("app.db")))


def init_db(db_path=DB_PATH):
    conn = sqlite3.connect(db_path)
    try:
        conn.execute(
            "CREATE TABLE IF NOT EXISTS tasks (id INTEGER PRIMARY KEY, title TEXT NOT
NULL, status TEXT DEFAULT 'new')"
        )
        conn.commit()
    finally:
        conn.close()


if __name__ == "__main__":
    init_db()
    print("База данных и таблица tasks успешно созданы.")
    print(f"Путь: {DB_PATH.resolve()}")
```